| 500 | 279 | 55.80 | 1.14 | Invalid | Employment_Flag |
|---|---|---|---|---|---|
| Total Records | Affected Rows | Issue Rate % | Avg Issues / Affected Row | Top Issue Type | Top Issue Field |

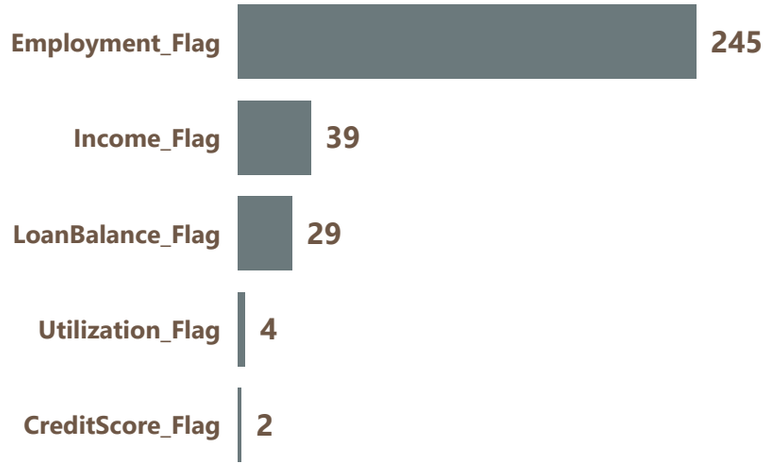
# Raw Data Quality Analysis & Insights

**"Record-level and cell-level data quality assessment"**

## Data Quality Insights

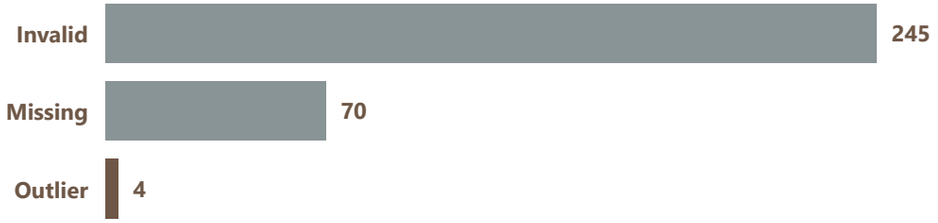
- **319 issues** detected across **500 records (2,500 cells).**
- **Invalid values dominate (76.8%)**, driven by inconsistent Employment Status (e.g., "EMP", "retired").
- Issues are uniformly distributed across locations, indicating a system-wide validation gap.
- **Priority action:** Standardize Employment Status inputs using controlled values.

## Top Fields Driving Data Quality Issues




## Issue Type Distribution

Record-Level & Field-Level Data Quality Assessment




Location

All

Payment Behavior

All

## Data Quality Remediation Framework

| Priority | Issue_Type | Governance_Owner | Remediation_Action |
|---|---|---|---|
| High | Invalid | Data Governance Team | Align with business rules or external reference data |
| High | Invalid | Credit Bureau Integration | Normalize scoring scale (e.g. 300–850) |
| High | Invalid | HR/Data Entry Team | Standardize Employment_Status values (use controlled taxono |
| High | Invalid | HR Team | Validate against realistic age ranges (e.g. 18–100) |
| High | Outlier | Risk/Compliance Team | Apply thresholds and investigate anomalies in source system |
| Low | Missing | Customer Service | Estimate based on account creation date or flag for review |

| 319 | 0 | 500 |
|---|---|---|
| Issues (Before) | Issues (After) | Total Record |

# Clean Data Quality Analysis & Insights

## RowQuality

- ☑ Select all
- ☑ Corrected (1)
- ☑ Corrected (2)
- ☑ Issue
- ☑ OK

## Clean Data Insight

**Detection:** Invalid ages, out-of-range credit scores, and inconsistent Employment Status values were flagged.
**Correction:** Median imputation, capping utilization (0–1), and standardized mappings were applied.
**Flagging**: Records classified as *OK*, *Corrected (n)*, or *Issue*.
**Impact:** 319 cell issues across 500 records, affecting 279 rows. After remediation, issue **rate dropped from 55.8% to near-zero**, strengthening governance.

## Affected Rows Breakdown

| RowQuality | Count of RowQuality |
|---|---|
| Corrected (1) | 213 |
| Corrected (2) | 2 |
| Issue | 64 |
| **Affected Rows** | **279** |

| Customer_ID | RowQuality | Employment_status_Flag | Income_Flag | LoanBalance_Flag | Age_Flag | Card_Flag | CreditScore_Flag | CU_Flag | Delinquent_Flag | DT |
|---|---|---|---|---|---|---|---|---|---|---|
| CUST0001 | Corrected (1) | Corrected | OK | OK | OK | OK | OK | OK | OK | OK |
| CUST0002 | OK | OK | OK | OK | OK | OK | OK | OK | OK | OK |
| CUST0003 | OK | OK | OK | OK | OK | OK | OK | OK | OK | OK |
| CUST0004 | OK | OK | OK | OK | OK | OK | OK | OK | OK | OK |
| CUST0005 | OK | OK | OK | OK | OK | OK | OK | OK | OK | OK |
| CUST0006 | OK | OK | OK | OK | OK | OK | OK | OK | OK | OK |
| CUST0007 | Corrected (1) | Corrected | OK | OK | OK | OK | OK | OK | OK | OK |
| CUST0008 | Corrected (1) | Corrected | OK | OK | OK | OK | OK | OK | OK | OK |

# Data_Quality_Clean


<?xml version="1.0" encoding="utf-8"?><Mashup xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns="http://schemas.microsoft.com/DataMashup"><Client>EXCEL</Client><Version>2.151.328.0</Version><MinVersion>2.21.0.0</MinVersion><Culture>en-IN</Culture><SafeCombine>true</SafeCombine><Items><Query Name="Clean_Data"><Formula><![CDATA[let // 1?? SOURCE Source = Excel.CurrentWorkbook(){[Name="Table5"]}[Content], BufferedSource = Table.Buffer(Source), // 2?? TYPE CONVERSION #"Changed Type" = Table.TransformColumnTypes(BufferedSource,{ {"Customer_ID", type text}, {"Age", Int64.Type}, {"Income", Int64.Type}, {"Credit_Score", Int64.Type}, {"Credit_Utilization", type number}, {"Missed_Payments", Int64.Type}, {"Delinquent_Account", Int64.Type}, {"Loan_Balance", Int64.Type}, {"Debt_to_Income_Ratio", type number}, {"Employment_Status", type text}, {"Account_Tenure", Int64.Type}, {"Credit_Card_Type", type text}, {"Location", type text} } & List.Transform( List.Select(Table.ColumnNames(Source), each Text.StartsWith(_, "Month_")), each {_, type text} )), // 3?? STATISTICS MedianIncome = List.Median(List.RemoveNulls(#"Changed Type"[Income])), MeanCreditScore = List.Average(List.RemoveNulls(#"Changed Type"[Credit_Score])), MedianLoanBalance = List.Median(List.RemoveNulls(#"Changed Type"[Loan_Balance])), MedianMissedPayments = List.Median(List.RemoveNulls(#"Changed Type"[Missed_Payments])), // 4?? RAW ? CLEAN ? FLAG IncomeClean = Table.AddColumn(#"Changed Type", "Income_Clean", each if [Income]=null then MedianIncome else [Income]), IncomeFlag = Table.AddColumn(IncomeClean, "Income_Flag", each if [Income]=null then "Median" else "OK"), CreditScoreClean = Table.AddColumn(IncomeFlag, "CreditScore_Clean", each if [Credit_Score]=null then MeanCreditScore else if [Credit_Score]<300 then 300 else if [Credit_Score]>850 then 850 else [Credit_Score]), CreditScoreFlag = Table.AddColumn(CreditScoreClean, "CreditScore_Flag", each if [Credit_Score]=null then "Mean" else if [Credit_Score]<300 or [Credit_Score]>850 then "Corrected" else "OK"), AgeClean = Table.AddColumn(CreditScoreFlag, "Age_Clean", each if [Age]<18 then 18 else if [Age]>100 then 100 else [Age]), AgeFlag = Table.AddColumn(AgeClean, "Age_Flag", each if [Age]<18 or [Age]>100 then "Corrected" else "OK"), CUClean = Table.AddColumn(AgeFlag, "CU_Clean", each if [Credit_Utilization]<0 then 0 else if [Credit_Utilization]>1 then 1 else [Credit_Utilization]), CUFlag = Table.AddColumn(CUClean, "CU_Flag", each if [Credit_Utilization]<0 or [Credit_Utilization]>1 then "Corrected" else "OK"), MissedPaymentsClean = Table.AddColumn(CUFlag, "MissedPayments_Clean", each if [Missed_Payments]=null then MedianMissedPayments else if [Missed_Payments]<0 then 0 else if [Missed_Payments]>12 then 12 else [Missed_Payments]), MissedPaymentsFlag = Table.AddColumn(MissedPaymentsClean, "MissedPayments_Flag", each if [Missed_Payments]=null then "Median" else if [Missed_Payments]<0 or [Missed_Payments]>12 then "Corrected" else "OK"), LoanBalanceClean = Table.AddColumn(MissedPaymentsFlag, "LoanBalance_Clean", each if [Loan_Balance]=null then MedianLoanBalance else [Loan_Balance]), LoanBalanceFlag = Table.AddColumn(LoanBalanceClean, "LoanBalance_Flag", each if [Loan_Balance]=null then "Median" else "OK"), DelinquentClean = Table.AddColumn(LoanBalanceFlag, "Delinquent_Clean", each if [Delinquent_Account]<0 then 0 else if [Delinquent_Account]>1 then 1 else [Delinquent_Account]), DelinquentFlag = Table.AddColumn(DelinquentClean, "Delinquent_Flag", each if [Delinquent_Account]<0 or [Delinquent_Account]>1 then "Corrected" else "OK"), DTIFlag = Table.AddColumn(DelinquentFlag, "DTI_Flag", each if [Debt_to_Income_Ratio]=null then "Corrected" else if [Debt_to_Income_Ratio]<0 or [Debt_to_Income_Ratio]>1 then "OutOfRange" else "OK"), AgeTenureFlag = Table.AddColumn(DTIFlag, "Age_Tenure_Flag", each if [Account_Tenure]>[Age_Clean] then "Invalid" else "OK"), // 5?? DYNAMIC MONTH CLEANING MonthCols = List.Select(Table.ColumnNames(#"Changed Type"), each Text.StartsWith(_, "Month_")), MonthCleaned = List.Accumulate( MonthCols, AgeTenureFlag, (state, current) => let merged = Table.NestedJoin(state, {current}, Mapping_Month, {"Raw Value"}, "Map", JoinKind.LeftOuter), expanded = Table.ExpandTableColumn(merged, "Map", {"Clean Value","Flag_Result"}, {current & "_Clean", current & "_Flag"}) in expanded ), // 6?? LOCATION LocationJoin = Table.NestedJoin(MonthCleaned, {"Location"}, Mapping_Location, {"Raw Value"}, "Map_Loc", JoinKind.LeftOuter), LocationExpanded = Table.ExpandTableColumn(LocationJoin, "Map_Loc", {"Clean Value","Flag_Result"}, {"Location_Clean","Location_Flag"}), // 7?? CARD CardJoin = Table.NestedJoin(LocationExpanded, {"Credit_Card_Type"}, Mapping_Card, {"Raw Value"}, "Map_Card", JoinKind.LeftOuter), CardExpanded = Table.ExpandTableColumn(CardJoin, "Map_Card", {"Clean Value","Flag_Result"}, {"Card_Clean","Card_Flag"}), // 8?? EMPLOYMENT STATUS EmploymentJoin = Table.NestedJoin(CardExpanded, {"Employment_Status"}, Employment_status_mapping, {"Raw Value"}, "Map_Emp", JoinKind.LeftOuter), EmploymentExpanded = Table.ExpandTableColumn(EmploymentJoin, "Map_Emp", {"Clean Value","Flag_Result"}, {"Employment_status_Clean","Employment_status_Flag"}), // 9?? ROW QUALITY RowQuality = Table.AddColumn(EmploymentExpanded, "RowQuality", each let currentRow = _, monthFlags = List.Transform(MonthCols, (m) => Record.Field(currentRow, m & "_Flag")), flags = { currentRow[Income_Flag], currentRow[CreditScore_Flag], currentRow[Age_Flag], currentRow[CU_Flag], currentRow[MissedPayments_Flag], currentRow[LoanBalance_Flag], currentRow[Delinquent_Flag], currentRow[DTI_Flag], currentRow[Age_Tenure_Flag], currentRow[Employment_status_Flag], currentRow[Card_Flag], currentRow[Location_Flag] } & monthFlags, hasKnownIssue = List.Contains(flags,"Invalid") or List.Contains(flags,"OutOfRange"), correctionCount = List.Count(List.Select(flags, each _ <> "OK")), hasUnknownFlag = List.Count(List.Select(flags, each _ <> "OK" and _ <> "Invalid" and _ <> "OutOfRange" and not Text.StartsWith(_, "Corrected"))) > 0 in if hasKnownIssue or hasUnknownFlag then "Issue" else if correctionCount>0 then "Corrected (" & Text.From(correctionCount) & ")" else "OK" ), // 10?? REORDER COLUMNS #"Reordered Columns" = Table.ReorderColumns(RowQuality, List.Combine({ {"Customer_ID"}, {"Income","Income_Clean","Income_Flag"}, {"Credit_Score","CreditScore_Clean","CreditScore_Flag"}, {"Age","Age_Clean","Age_Flag"}, {"Credit_Utilization","CU_Clean","CU_Flag"}, {"Missed_Payments","MissedPayments_Clean","MissedPayments_Flag"}, {"Loan_Balance","LoanBalance_Clean","LoanBalance_Flag"}, {"Delinquent_Account","Delinquent_Clean","Delinquent_Flag"}, {"Debt_to_Income_Ratio","DTI_Flag"}, {"Account_Tenure","Age_Tenure_Flag"}, {"Employment_Status","Employment_status_Clean","Employment_status_Flag"}, {"Credit_Card_Type","Card_Clean","Card_Flag"}, {"Location","Location_Clean","Location_Flag"}, List.Combine(List.Transform(MonthCols, each {_, _ & "_Clean", _ & "_Flag"})) }) ) in #"Reordered Columns"]]></Formula><LoadToWorksheet>true</LoadToWorksheet><FillObjectType>Table</FillObjectType><LoadToDataModel>false</LoadToDataModel><IsParameterQuery>false</IsParameterQuery><IsDirectQuery xsi:nil="true" /></Query><Query Name="Mapping_Month"><Formula><![CDATA[let Source = Excel.CurrentWorkbook(){[Name="Mapping_Month"]}[Content], #"Changed Type" = Table.TransformColumnTypes(Source,{{"Raw Value", type text}, {"Clean Value", type text}, {"Flag_Result", type text}}) in #"Changed Type"]]></Formula><LoadToWorksheet>false</LoadToWorksheet><FillObjectType>ConnectionOnly</FillObjectType>

# Fact_Table_Flagged

<?xml version="1.0" encoding="utf-8"?><Mashup xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns="http://schemas.microsoft.com/DataMashup"><Client>EXCEL</Client><Version>2.151.328.0</Version><MinVersion>2.21.0.0</MinVersion><Culture>en-IN</Culture><SafeCombine>true</SafeCombine><Items><Query Name="Flaged_Table"><Formula><![CDATA[// Flaged_Table let // ========== // 0. Load Fact Table // ========== Source = Excel.CurrentWorkbook(){[Name="Fact_Table"]}[Content], Typed = Table.TransformColumnTypes(Source,{ {"Customer_ID", type text}, {"Age", Int64.Type}, {"Income", Int64.Type}, {"Credit_Score", Int64.Type}, {"Credit_Utilization", type number}, {"Missed_Payments", Int64.Type}, {"Delinquent_Account", Int64.Type}, {"Loan_Balance", Int64.Type}, {"Debt_to_Income_Ratio", type number}, {"Employment_Status", type text}, {"Account_Tenure", Int64.Type}, {"Credit_Card_Type", type text}, {"Location", type text}, {"Month_1", type text}, {"Month_2", type text}, {"Month_3", type text}, {"Month_4", type text}, {"Month_5", type text}, {"Month_6", type text} }), // ========== // 1. Validation Tables // ========== EmploymentValidation = Excel.CurrentWorkbook(){[Name="Employment_Status"]}[Content], CreditCardValidation = Excel.CurrentWorkbook(){[Name="Credit_Card_Type"]}[Content], LocationValidation = Excel.CurrentWorkbook(){[Name="Location"]}[Content], MonthValidation = Excel.CurrentWorkbook(){[Name="Month"]}[Content], // ========== // 2. Numeric Flags // ========== AddIncomeFlag = Table.AddColumn(Typed, "Income_Flag", each if [Income] = null then "Missing: Income" else if [Income] < 0 then "Outlier: <0" else if [Income] > 1000000 then "Outlier: >1M" else "OK: 0–1M", type text), AddAgeFlag = Table.AddColumn(AddIncomeFlag, "Age_Flag", each if [Age] = null then "Missing: Age" else if [Age] < 18 then "Outlier: <18" else if [Age] > 100 then "Outlier: >100" else "OK: 18–100", type text), AddCreditScoreFlag = Table.AddColumn(AddAgeFlag, "CreditScore_Flag", each if [Credit_Score] = null then "Missing: Score" else if [Credit_Score] < 300 then "Outlier: <300" else if [Credit_Score] > 850 then "Outlier: >850" else "OK: 300–850", type text), AddLoanBalanceFlag = Table.AddColumn(AddCreditScoreFlag, "LoanBalance_Flag", each if [Loan_Balance] = null then "Missing: Loan" else if [Loan_Balance] < 0 then "Outlier: <0" else "OK: =0", type text), AddUtilizationFlag = Table.AddColumn(AddLoanBalanceFlag, "Utilization_Flag", each if [Credit_Utilization] = null then "Missing: Utilization" else if [Credit_Utilization] < 0 then "Outlier: <0" else if [Credit_Utilization] > 1 then "Outlier: >1" else "OK: 0–1", type text), AddDTIFlag = Table.AddColumn(AddUtilizationFlag, "DTI_Flag", each if [Debt_to_Income_Ratio] = null then "Missing: DTI" else if [Debt_to_Income_Ratio] < 0 then "Outlier: <0" else if [Debt_to_Income_Ratio] > 1 then "Outlier: >1" else "OK: 0–1", type text), // ========== // Missed Payments // ========== MaxMissed = List.Max(Typed[Missed_Payments]), AddMissedPaymentsFlag = Table.AddColumn(AddDTIFlag, "MissedPayments_Flag", each if [Missed_Payments] = null then "Missing: Missed" else if [Missed_Payments] < 0 then "Outlier: <0" else if [Missed_Payments] > MaxMissed then "Outlier: >" & Text.From(MaxMissed) else "OK: 0–" & Text.From(MaxMissed), type text), AddTenureFlag = Table.AddColumn(AddMissedPaymentsFlag, "Tenure_Flag", each if [Account_Tenure] = null then "Missing: Tenure" else if [Age] <> null and [Account_Tenure] > [Age] then "Outlier: >Age" else "OK: =Age", type text), // ========== // Contextual Checks // ========== AddContextualFlag = Table.AddColumn(AddTenureFlag, "Contextual_Flag", each let flags = { if [Account_Tenure] <> null and [Age] <> null and [Account_Tenure] > [Age] then "Error: Tenure>Age" else null, if [Loan_Balance] <> null and [Income] <> null and [Loan_Balance] > [Income] * 10 then "Error: Loan>>Income" else null, if [Income] <> null and [Income] = 0 and [Employment_Status] = "Employed" then "Error: Income=0+Employed" else null, if [Debt_to_Income_Ratio] <> null and [Debt_to_Income_Ratio] > 1 then "Error: DTI>1" else null }, nonNullFlags = List.RemoveNulls(flags) in if List.Count(nonNullFlags)=0 then "OK: Context" else Text.Combine(nonNullFlags,"; "), type text), // ========== // Categorical Validation // ========== MergeEmployment = Table.NestedJoin(AddContextualFlag, {"Employment_Status"}, EmploymentValidation, {"Raw Data"}, "ValidationData", JoinKind.LeftOuter), ExpandEmployment = Table.ExpandTableColumn(MergeEmployment, "ValidationData", {"Validation"}), AddEmploymentFlag = Table.AddColumn(ExpandEmployment, "Employment_Flag", each if [Employment_Status]=null then "Missing: Employment" else if [Validation]="Valid" then "OK: Employment" else "Invalid: " & [Employment_Status]), CleanEmployment = Table.RemoveColumns(AddEmploymentFlag,{"Validation"}), MergeCredit = Table.NestedJoin(CleanEmployment, {"Credit_Card_Type"}, CreditCardValidation, {"Raw Data"}, "ValidationData", JoinKind.LeftOuter), ExpandCredit = Table.ExpandTableColumn(MergeCredit,"ValidationData",{"Validation"}), AddCreditFlag = Table.AddColumn(ExpandCredit,"CreditCard_Flag", each if [Credit_Card_Type]=null then "Missing: Card" else if [Validation]="Valid" then "OK: Card" else "Invalid: " & [Credit_Card_Type]), CleanCredit = Table.RemoveColumns(AddCreditFlag,{"Validation"}), MergeLocation = Table.NestedJoin(CleanCredit, {"Location"}, LocationValidation, {"Raw Data"}, "ValidationData", JoinKind.LeftOuter), ExpandLocation = Table.ExpandTableColumn(MergeLocation,"ValidationData",{"Validation"}), AddLocationFlag = Table.AddColumn(ExpandLocation,"Location_Flag", each if [Location]=null then "Missing: Location" else if [Validation]="Valid" then "OK: Location" else "Invalid: " & [Location]), CleanLocation = Table.RemoveColumns(AddLocationFlag,{"Validation"}), // ========== // Month Validation // ========== MonthColumns = List.Select(Table.ColumnNames(CleanLocation), each Text.StartsWith(_, "Month_")), FinalWithMonths = List.Accumulate( MonthColumns, CleanLocation, (state,col)=> let merged = Table.NestedJoin(state, {col},MonthValidation,{"Raw Data"},"ValidationData",JoinKind.LeftOuter), expanded = Table.ExpandTableColumn(merged,"ValidationData",{"Validation"}), flagged = Table.AddColumn(expanded,col & "_Flag", each if Record.Field(_,col)=null then "Missing: " & col else if [Validation]="Valid" then "OK: Month" else "Invalid: " & col & "=" & Record.Field(_,col)), cleaned = Table.RemoveColumns(flagged,{"Validation"}) in cleaned ), // ========== // Row Quality // ========== AddRowQuality = Table.AddColumn(FinalWithMonths,"Row_Quality", each let flagColumns = List.Select(Table.ColumnNames(FinalWithMonths), each Text.EndsWith(_, "_Flag")), badFlags = List.Select(flagColumns,(c)=> not Text.StartsWith(Record.Field(_,c),"OK")), issueCount = List.Count(badFlags) in if issueCount=0 then "Row OK" else if issueCount=1 then "1 issue" else Text.From(issueCount) & " issues" ), AddIssueDetails = Table.AddColumn(AddRowQuality,"Issue_Details", each let flagColumns = List.Select(Table.ColumnNames(AddRowQuality), each Text.EndsWith(_, "_Flag")), badFlags = List.Select(flagColumns,(c)=> not Text.StartsWith(Record.Field(_,c),"OK")), badFlagsClean = List.Transform(badFlags, each Text.BeforeDelimiter(_, "_Flag")) in if List.Count(badFlagsClean)=0 then "No issues" else Text.Combine(badFlagsClean,"; ") ), // ========== // Sort // ========